

Продвинутое машинное обучение

Лекция 3: Bagging / RF / ExtraTrees: снижение дисперсии, ООВ-оценка, рандомизация

Автор курса: Лапшина Анастасия (University of Maryland, College Park, ВШЭ)

Одиночное дерево интерпретируемо,
но нестабильно, жадно, имеет
высокую дисперсию и склонно к
переобучению. Что делать?

Часть 1. Bagging

Bootstrap-выборки и агрегирование

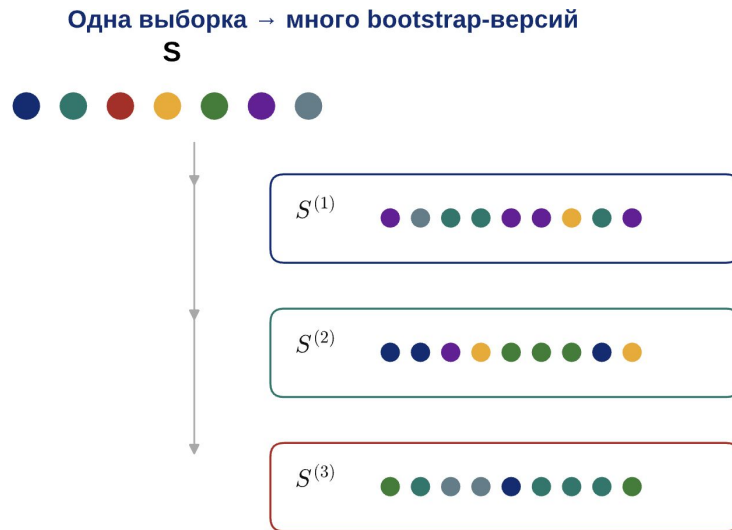
Одиночное дерево сильно зависит от ранних сплитов → предсказание будет меняться от выборки к выборке.

Идея: обучить много деревьев на разных bootstrap-выборках и агрегировать их предсказания

$$S = \{z_i = (x_i, y_i)\}_{i=1}^n$$

$$S^{(b)} = \{z_1^{(b)}, \dots, z_n^{(b)}\}, \quad z_i^{(b)} \sim \widehat{P}_S$$

- извлекаем объекты с возвращением
(объект может повторяться, либо не попасть в выборку совсем)



каждая bootstrap-выборка обучает своё дерево

Bagging

На каждой бутстрап выборке обучаем модель

$$\hat{f}^{(b)} = A(S^{*(b)}),$$

а затем усредняем прогноз (регрессия)

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x)$$

В задаче классификации:

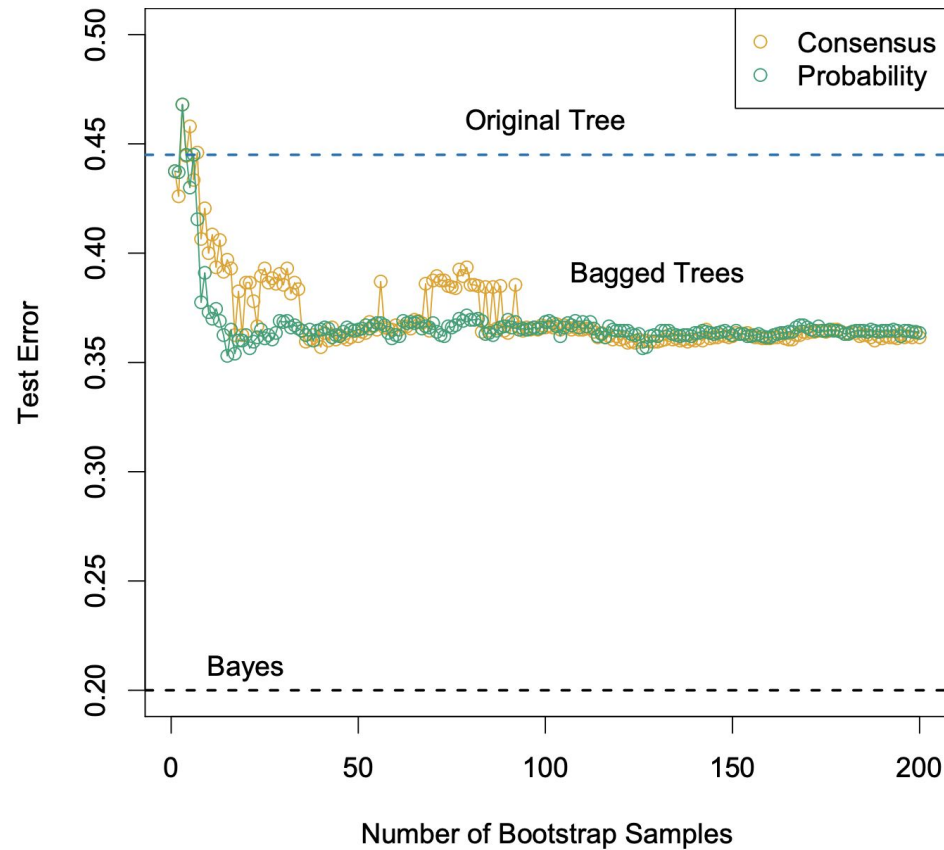
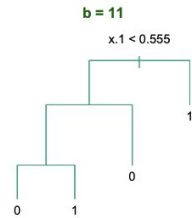
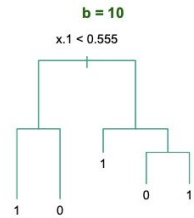
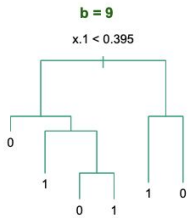
- Усреднение

$$\hat{p}_k^{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{p}_k^{(b)}(x), \quad k = 1, \dots, K$$

$$\hat{y}_{\text{bag}}(x) = \arg \max_{k=1, \dots, K} \hat{p}_k^{\text{bag}}(x)$$

- Жесткое голосование

$$\hat{y}_{\text{bag}}(x) = \text{mode}(\hat{y}^{(1)}(x), \dots, \hat{y}^{(B)}(x))$$



С чем борется бэггинг

Усреднение работает не на обе составляющие ошибки сразу. Разберём их по отдельности:

Variance - падает

Усредняем B моделей, разброс среднего меньше разброса одной модели

Bias - почти не меняется

$$\mathbb{E}[\bar{Z}_B] = \frac{1}{B} \sum_{b=1}^B \mathbb{E}[Z_b] = \mu$$

Среднее моделей смещено ровно так же, как одна модель. Усреднение bias не убирает.

Отсюда следует, какое дерево брать базовой моделью:

Мелкое или обрезанное дерево

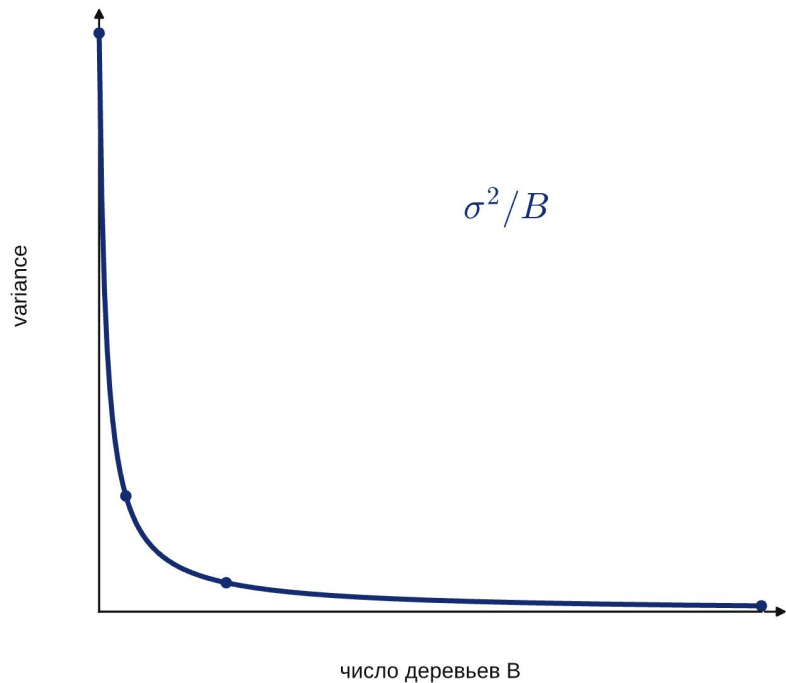
bias $\uparrow$, variance $\downarrow$ — усреднять почти нечего, а bias так и останется

Глубокое дерево без pruning

bias $\downarrow$, variance $\uparrow$ — то, с чем нам поможет бэггинг, потому что умеет с этим работать

Поэтому в bagging и Random Forest деревья растут до конца. Pruning здесь не нужен и даже вреден: он добавит bias, который ансамбль не исправит.

Независимые модели



В идеальном случае (независимость моделей) variance среднего уменьшается пропорционально числу деревьев B

$$\bar{Z}_B = \frac{1}{B} \sum_{b=1}^B Z_b$$

$$\mathbb{E}[Z_b] = \mu, \quad \text{Var}(Z_b) = \sigma^2$$

$$\text{Var}(\bar{Z}_B) = \text{Var}\left(\frac{1}{B} \sum_{b=1}^B Z_b\right) = \frac{1}{B^2} \sum_{b=1}^B \text{Var}(Z_b) = \frac{\sigma^2}{B}$$

Reality check

В реальности bootstrap-деревья не независимы:

$$\text{Var}(Z_b(x)) = \sigma^2, \quad \text{Corr}(Z_b(x), Z_{b'}(x)) = \rho$$

$$\text{Var}(\bar{Z}_B(x)) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

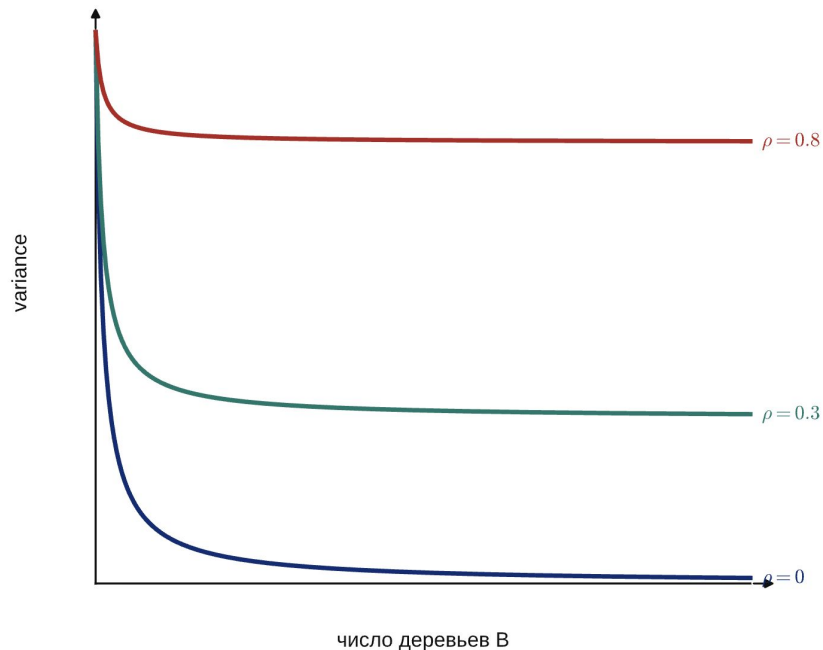
$$\rho\sigma^2$$

не исчезает при росте B

$$\frac{1-\rho}{B}\sigma^2$$

уменьшается при росте B

ρ задаёт нижнюю границу variance



Откуда берётся $\rho\sigma^2$

Повторим то же вычисление, но уже без предположения независимости деревьев:

$$\text{Var}(Z_b) = \sigma^2, \quad \text{Cov}(Z_b, Z_{b'}) = \rho\sigma^2 \quad (b \neq b')$$

- 1 Дисперсия суммы – это сумма всех элементов ковариационной матрицы:

$$\text{Var}(\bar{Z}_B) = \frac{1}{B^2} \text{Var}\left(\sum_{b=1}^B Z_b\right) = \frac{1}{B^2} \left[\sum_b \text{Var}(Z_b) + \sum_{b \neq b'} \text{Cov}(Z_b, Z_{b'}) \right]$$

- 2 Диагональных элементов B , внедиагональных – $B(B-1)$:

$$= \frac{1}{B^2} \left[\underbrace{B \sigma^2}_{B \text{ диагональных}} + \underbrace{B(B-1) \rho\sigma^2}_{B(B-1) \text{ внедиагональных}} \right]$$

- 3 Раскрываем и перегруппировываем:

$$= \frac{\sigma^2}{B} + \frac{B-1}{B} \rho\sigma^2 = \rho\sigma^2 + \frac{1-\rho}{B} \sigma^2$$

Первое слагаемое собрано из $B(B-1)$ ковариаций – их почти столько же, сколько B^2 , поэтому деление на B^2 его не убирает.

При $\rho = 0$ внедиагональных членов нет, и остаётся σ^2 / B .

$$B \rightarrow \infty: \quad \text{Var}(\bar{Z}_B(x)) \rightarrow \rho\sigma^2$$

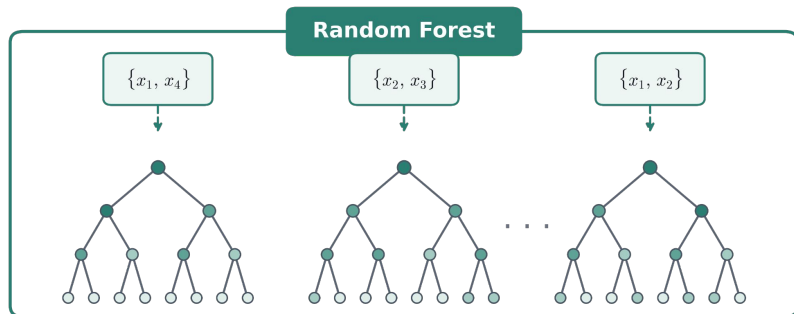
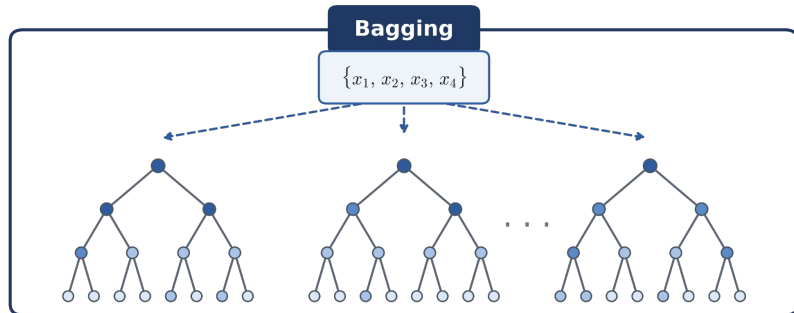
Bagging снижает variance через усреднение, но коррелированная часть ошибки остаётся



Следующий шаг: декоррелировать деревья

Часть 2. Random Forest и ООВ

От Bagging к Random Forest



Идея: в каждом узле ищем split только среди случайного подмножества признаков

$$\text{Var}(\bar{T}_B(x)) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$



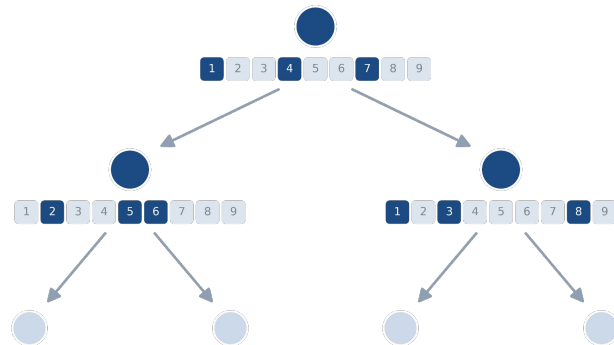
Random Forest

дополнительно уменьшает ρ
за счёт случайного выбора
признаков

Алгоритм RF



Одно дерево RF: $m = 3$ признака из $p = 9$



 выбранные m  остальные $p - m$
у каждого узла свой случайный набор признаков

RF = Bagging + случайный выбор признаков в узлах для разбиения

Сколько признаков брать в узле

m (`max_features`) – главный гиперпараметр Random Forest. Он управляет тем p , которое стоит в формуле дисперсии:

$m = p \implies$ обычный bagging, $m = 1 \implies$ сплит по случайному признаку

m ↓ – меньше признаков в узле

Деревья меньше похожи друг на друга: $p \downarrow$, а с ним и предел $\rho\sigma^2$. Но выбор в каждом узле беднее, отдельное дерево слабее – bias растёт.

m ↑ – больше признаков в узле

Каждое дерево сильнее, bias ниже. Но все деревья цепляются за одни и те же сильные признаки: $p \uparrow$, и нижняя граница дисперсии поднимается.

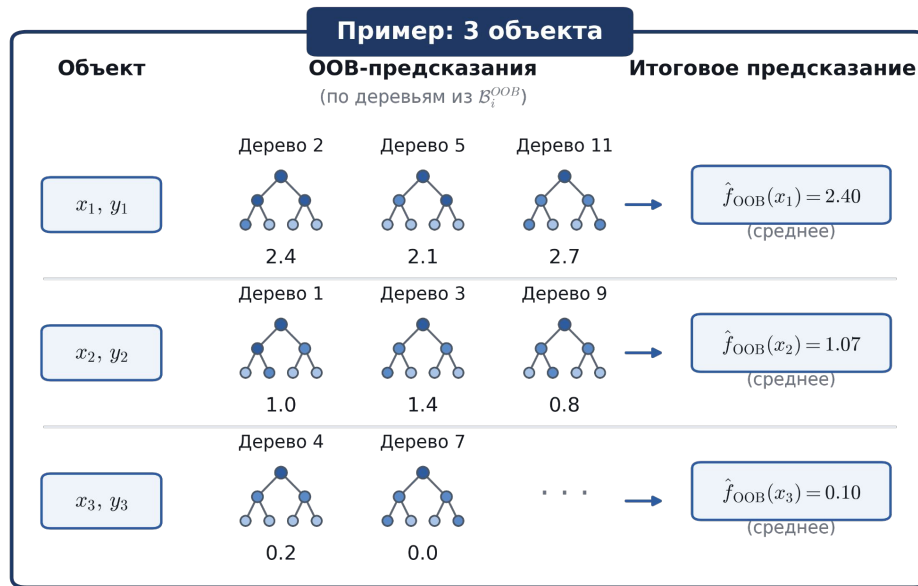
Значения по умолчанию

Классическая рекомендация Бреймана: $\sqrt{p}$ для классификации и $p / 3$ для регрессии. В scikit-learn у классификатора `max_features='sqrt'`, а у регрессора по умолчанию берутся все признаки – то есть это уже не Random Forest, а обычный bagging. Для регрессии **m** почти всегда стоит задавать руками.

Подбирать **m удобно по ООВ-кривой, не тратя на это ресурсы отдельной валидации**

OOB samples: важная фишка Random Forest

1. Для каждого объекта (x_i, y_i) находим набор деревьев $\mathcal{B}_i^{OOB}$
2. Строим OOB-предсказание только по этим деревьям
3. Сравниваем OOB-предсказание с истинным y_i
4. Усредняем ошибки по всем объектам



Почему удобно: OOB дает оценку качества близкую к валидации и помогает выбирать число деревьев БЕЗ отдельной валидации

ООВ-объекты: откуда берётся 36.8%

1. Обучающая выборка состоит из n объектов, бутстрап-выборка $S^{(b)}$ — тоже из n объектов, но с повторениями.
2. Для одного объекта вероятность не быть выбранным за одно извлечение:

$$P(\text{не выбрать } z_i \text{ за одно извлечение}) = 1 - \frac{1}{n}$$

3. Извлечений n , они независимы — перемножаем:

$$P(z_i \notin S^{(b)}) = \left(1 - \frac{1}{n}\right)^n$$

$$\left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368 \quad (n \rightarrow \infty)$$

$$P(z_i \in S^{(b)}) = 1 - \left(1 - \frac{1}{n}\right)^n \approx 0.632$$

Интерпретация

- примерно 36.8% объектов не попадают в bootstrap-выборку данного дерева
- эти объекты — out-of-bag (OOB) для данного дерева
- именно по ним строят OOB-предсказание объекта

Когда OOB-оценке нельзя верить

OOB - дешёвая замена валидации, но существуют ситуации, где оценка перестаёт быть корректной и честной:

Мало деревьев

Объект предсказывают только те деревья, для которых он OOB, - в среднем $\approx 0,368 \cdot B$ деревьев. При $B = 50$ это ≈ 18 деревьев: оценка шумная и пессимистичная, она измеряет качество маленького леса, а не вашего.

Группы в данных

Если один клиент, пациент или сессия встречается в выборке несколько раз, копии почти наверняка попадут в bootstrap. OOB ломается ровно так же, как обычный KFold без групп.

Время

Bootstrap перемешивает наблюдения, поэтому дерево обучается на будущем относительно своего OOB-объекта. PIT нарушен, оценка завышена - нужен временной split.

Итоговое правило: OOB-валидация корректна только для независимых (IID) данных без временной структуры при условии достаточного количества деревьев в ансамбле. Во всех остальных случаях используйте специализированные разбиения (*GroupKFold*, *TimeSeriesSplit*)

Важность признаков в Random Forest

Impurity-based (MDI)

$$\text{MDI}(j) = \frac{1}{B} \sum_{b=1}^B \sum_{A \in T_b : j_A = j} \frac{|A|}{n} \Delta Q(A)$$

Суммарное уменьшение нечистоты по всем сплитам, сделанным по признаку j , усреднённое по деревьям. Считается бесплатно при обучении – это `feature_importances_` в `scikit-learn`.

Наследует **variable selection bias** из лекции 2: признак с большим числом кандидатов получает завышенную важность. И считается на `train`.

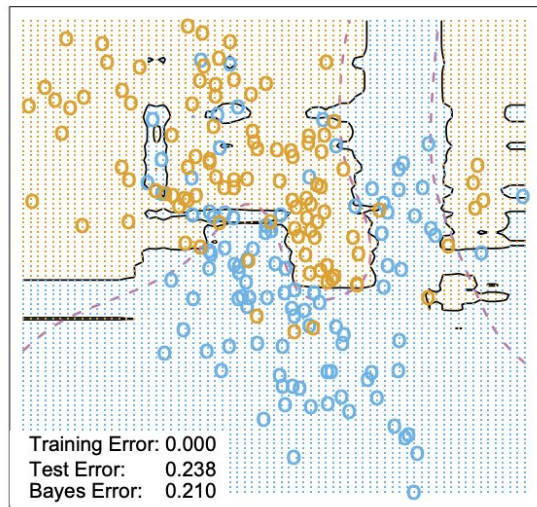
Permutation importance

$$\text{PI}(j) = \hat{R}(\hat{f}, D^{\pi_j}) - \hat{R}(\hat{f}, D)$$

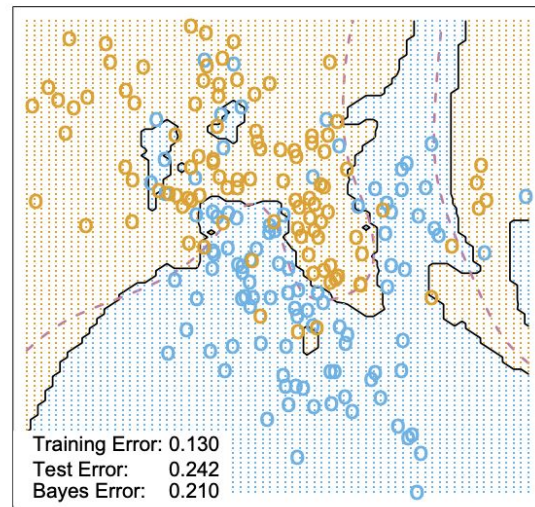
Перемешиваем значения признака и смотрим, насколько упало качество на отложенных или OOB-данных. От числа `split`-кандидатов не зависит: измеряет вклад в предсказание, а не в разбиение `train`.

Дороже. При сильно коррелированных признаках **размывает важность** между ними: перемешали один - второй компенсирует.

Random Forest Classifier



3-Nearest Neighbors



Сравнение Random Forest и 3-NN на mixture data. Поскольку отдельные деревья в Random Forest используют осе-ориентированные разбиения, итоговые области решений также сохраняют осе-ориентированный, “ступенчатый” характер.

Можем попробовать еще сильнее уменьшить корреляцию?
За счёт чего?

Часть 3. ExtraTrees

Декоррелируем сильнее

Bagging: bootstrap + averaging

Random Forest: random features + best threshold

ExtraTrees: random features + random threshold



Идем еще дальше: рандомизируем не только набор признаков, но и точки разбиения

Алгоритм ExtraTrees

1. В узле A случайно выбираем подмножество признаков $M_A \subset \{1, \dots, p\}$, $|M_A| = m$
2. Для каждого $j \in M_A$ генерируем случайный порог (threshold):

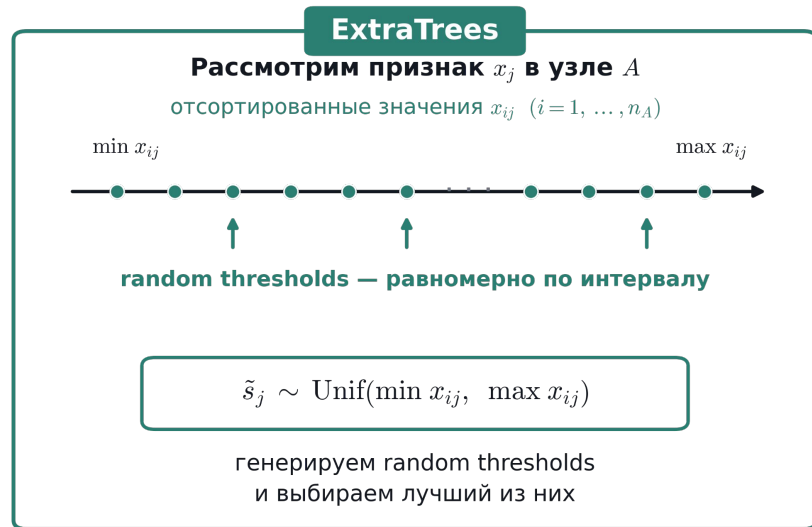
$$\tilde{s}_j \sim \text{Unif} \left(\min_{i \in A} x_{ij}, \max_{i \in A} x_{ij} \right)$$

3. Для каждого кандидата считаем уменьшение нечистоты:

$$\Delta Q(j, \tilde{s}_j; A) = Q(A) - \frac{|A_L|}{|A|} Q(A_L) - \frac{|A_R|}{|A|} Q(A_R)$$

4. Выбираем лучший среди случайных кандидатов:

$$j_A^* \in \operatorname{argmax}_{j \in M_A} \Delta Q(j, \tilde{s}_j; A), \quad x_{j_A^*} \leq \tilde{s}_{j_A^*}$$



Преимущества (и недостатки)

ExtraTrees ускоряет split search, потому что не выполняет exhaustive search по порогам → обучение дерева быстрее

Еще сильнее декоррелируем деревья → variance снижается

НО:

Рандомные пороги могут ослабить отдельные деревья → рост bias

Не всегда есть преимущество перед Random Forest на малых наборах данных

Часть 4. Нет предела совершенству

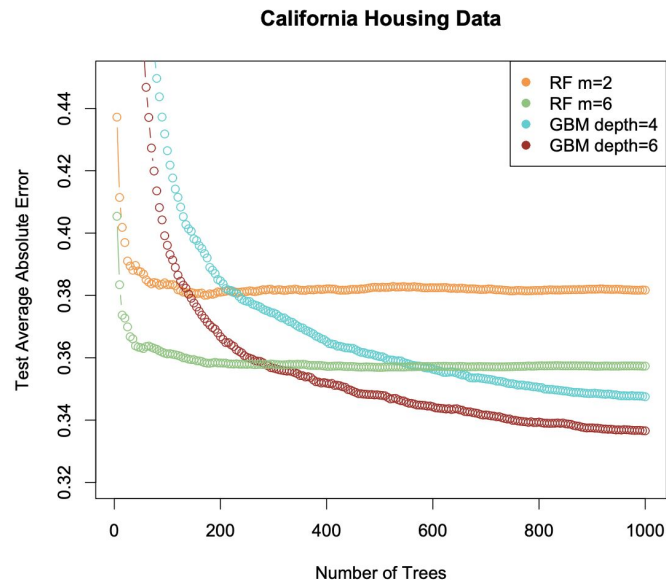
Что мы уже имеем

	Bootstrap	Случайный отбор фичей	Поиск порога	Идея/интуиция
Bagging	Да	Нет	Exhaustive search	Агрегируем глубокие нестабильные деревья
Random Forest	Да	Да	Exhaustive search	Снижаем корреляцию между деревьями
ExtraTrees	Обычно нет*	Да	Случайный, затем лучший из кандидатов	Еще сильнее декоррелируем

* в scikit-learn bootstrap=False по умолчанию

Что можем сделать

1. Будем строить деревья последовательно, а не параллельно
2. Каждое следующее дерево учитывает и исправляет ошибки предыдущего
3. Выстраиваем ансамбль, уменьшая bias и loss



Tricky question: являются ли DT/RF линейными моделями?

Дерево как линейная модель в новом признаковом пространстве

Дерево разбивает пространство на непересекающиеся области:

$$\mathcal{X} = \bigcup_{m=1}^M R_m, \quad R_m \cap R_{m'} = \emptyset \quad (m \neq m')$$

1 Индикаторы листьев:

$$\varphi_m(x) = \mathbb{1}\{x \in R_m\}, \quad m = 1, \dots, M$$

2 Вектор признаков-индикаторов:

$$\varphi(x) = (\varphi_1(x), \dots, \varphi_M(x))^\top$$

$$\hat{f}_{\text{tree}}(x) = \sum_{m=1}^M c_m \varphi_m(x) = c^\top \varphi(x)$$

модель линейна по индикаторам, но не по исходным признакам

Одно дерево (1):

$$\hat{f}^{(b)}(x) = \sum_{m=1}^{M_b} c_m^{(b)} \mathbb{1}\{x \in R_m^{(b)}\}$$

Random Forest (2):

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x)$$

Подставляем (1) в (2):

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B \sum_{m=1}^{M_b} c_m^{(b)} \mathbb{1}\{x \in R_m^{(b)}\}$$

двойная сумма: по деревьям и по листьям внутри каждого дерева

Что имеем в итоге

Обозначим индикаторы и веса:

$$\varphi_m^{(b)}(x) = \mathbb{1}\{x \in R_m^{(b)}\}, \quad \theta_m^{(b)} = \frac{c_m^{(b)}}{B}$$

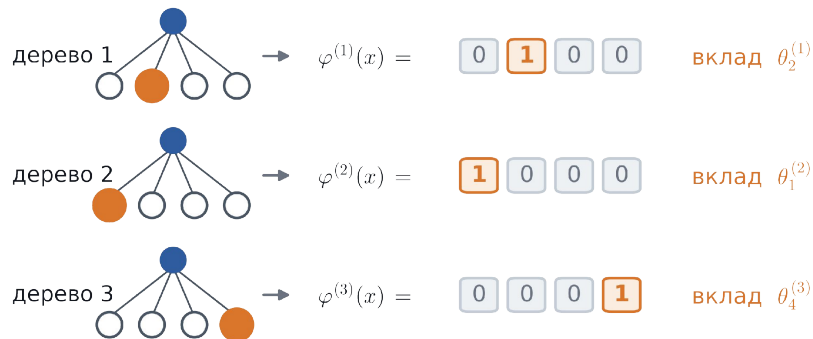
$$\hat{f}_{\text{RF}}(x) = \sum_{b=1}^B \sum_{m=1}^{M_b} \theta_m^{(b)} \varphi_m^{(b)}(x)$$

сумма по всем листьям всех деревьев

Вывод

Random Forest нелинеен в исходном пространстве X , но является линейной комбинацией индикаторов листьев всех деревьев.

для конкретного x в каждом дереве активен ровно один лист



$$\hat{f}_{\text{RF}}(x) = \theta_2^{(1)} + \theta_1^{(2)} + \theta_4^{(3)} + \dots$$

ИСТОЧНИКИ

1. Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer.
2. Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24, 123–140.
3. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
4. Breiman, L., Friedman, J. H., Olshen, R. A., Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
5. Geurts, P., Ernst, D., Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63, 3–42.
6. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
7. James, G., Witten, D., Hastie, T., Tibshirani, R., Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.